Structures de répétition

Pour exécuter à plusieurs reprises un bloc d'instruction, il existe deux structures de répétition :
• les boucles conditionnelles : le bloc est répété tant qu'une condition est vérifiée ;
• les boucles inconditionnelles : le bloc est répété un nombre donné de fois.

I) BOUCLES CONDITIONNELLES :

1°) Généralités :

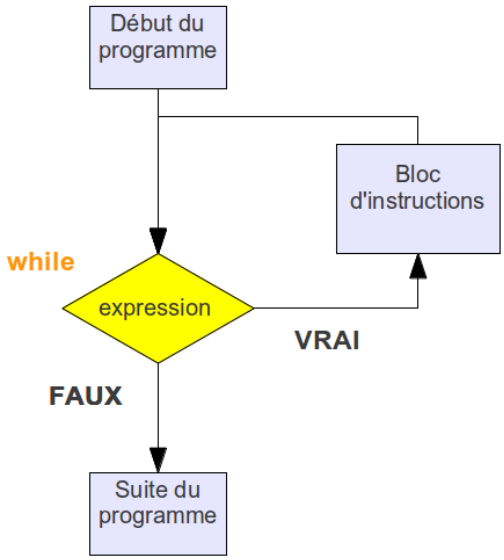
En python, une boucle conditionnelle est introduite par le mot clé while dont la syntaxe est :

```
while condition :
        bloc d'instructions à répéter tant que condition est vraie
```

Exemple : Le jeu du oui ou non :

```
reponse = ""
while reponse != "oui" and reponse != "non":
        reponse = input("Répondez oui ou non : ")
```

2°) L'instruction while :



Syntaxe

```
while expression:            # ne pas oublier le signe de ponctuation ':'
    bloc d'instructions      # attention à l'indentation
# suite du programme
```

Si l'expression est vraie (True) le bloc d'instructions est exécuté, puis l'expression est à nouveau évaluée.
Le cycle continue jusqu'à ce que l'expression soit fausse (False) : on passe alors à la suite du programme.

Exemple : un script qui compte de 1 à 4

```
# script Boucle1.py

# initialisation de la variable de comptage
compteur = 1
while compteur < 5:
   # ce bloc est exécuté tant que la condition (compteur < 5) est vraie
   print(compteur, compteur < 5)
   compteur += 1    # incrémentation du compteur, compteur = compteur + 1
```

```
print(compteur < 5)
print("Fin de la boucle")
>>>
1 True
2 True
3 True
4 True
False
Fin de la boucle
```

Table de multiplication par 8

```
# script Boucle2.py

compteur = 1            # initialisation de la variable de comptage
while compteur <= 10:
    # ce bloc est exécuté tant que la condition (compteur <= 10) est vraie
    print(compteur, '* 8 =', compteur*8)
    compteur += 1    # incrémentation du compteur, compteur = compteur + 1
print("Et voilà !")

>>>
1 * 8 = 8
2 * 8 = 16
3 * 8 = 24
4 * 8 = 32
5 * 8 = 40
6 * 8 = 48
7 * 8 = 56
8 * 8 = 64
9 * 8 = 72
10 * 8 = 80
Et voilà !
```

Affichage de l'heure courante

```
# script Boucle3.py

import time     # importation du module time
quitter = 'n'   # initialisation
while quitter != 'o':
    # ce bloc est exécuté tant que la condition est vraie
    # strftime() est une fonction du module time
    print('Heure courante ', time.strftime('%H:%M:%S'))
    quitter = input("Voulez-vous quitter le programme (o/n) ? ")
print("A bientôt")

>>>
Heure courante  09:48:54
Voulez-vous quitter le programme (o/n) ? n
Heure courante  09:48:58
Voulez-vous quitter le programme (o/n) ? o
A bientôt
```

<u>Exercice 1</u> : Écrire un programme qui demande à l'utilisateur de taper un nombre entre 0 et 100, et qui recommence tant que l'utilisateur tape autre chose. Le programme renverra le nombre finalement tapé.

<u>Exercice 2</u> : Écrire un programme qui renvoie le rang du dernier terme strictement positif de la suite $(u_n)_{n\in\mathbb{N}}$ définie par $u_{n+1} = (1/2)u_n - 3n$ en fonction de la valeur de u_0.

II) BOUCLES INCONDITIONNELLES :

1°) Généralités :

Lorsque l'on sait par avance le nombre de fois qu'un bloc doit être répété, on peut utiliser une boucle inconditionnelle, introduite par le mot clé for. En Python, la boucle for peut-être utilisée de plusieurs façons différentes sur lesquelles nous reviendrons plus tard. Pour l'instant, on utilisera la syntaxe :

```
for i in range(n):
        bloc à répéter n fois, i prenant successivement les valeurs 0, 1, ..., n-1.
```

Exemple : Calcul de la somme des termes d'une progression géométrique.

```
"""Renvoie la somme des n premiers termes d'une progression géométrique de raison q"""
s = 0
n = int(input("Nombre de termes : "))
q = float(input("Raison : "))

for k in range(n):
        s += q ** k
print("La somme des", n, "premiers termes est :", s)
```

Remarque : La fonction range peut prendre 1, 2 ou 3 arguments sous la formerange(fin) (pour . 0 ; fin-1.), range(debut, fin) (pour . debut ; fin-1.) ou range(debut, fin, pas) (pour { debut ; debut + pas ; debut + 2pas ; . . . ; fin-1 }).

2°) L'instruction for :

Syntaxe

```
for élément in séquence :
   bloc d'instructions
# suite du programme
```

Les éléments de la séquence sont issus d'une chaîne de caractères ou bien d'une liste.

Exemple avec une séquence de caractères

```
# script Boucle4.py

chaine = 'Bonsoir'
for lettre in chaine:                # lettre est la variable d'itération
   print(lettre)
print("Fin de la boucle")
```

La variable lettre est initialisée avec le premier élément de la séquence ('B').
Le bloc d'instructions est alors exécuté.
Puis la variable lettre est mise à jour avec le second élément de la séquence ('o') et le bloc d'instructions à nouveau exécuté...
Le bloc d'instructions est exécuté une dernière fois lorsqu'on arrive au dernier élément de la séquence ('r') :

```
>>>
B
o
n
s
o
i
r
Fin de la boucle
```

Exemple avec les éléments d'une liste

```
# script Boucle5.py

maliste = ['Pierre', 67.5, 18]
for element in maliste:
```

```
    print(element)
print("Fin de la boucle")
```

Là, on affiche dans l'ordre les éléments de la liste :

```
>>>
Pierre
67.5
18
Fin de la boucle
```

Fonction range()

L'association avec la fonction range() est très utile pour créer des séquences automatiques de nombres entiers :

```
# script Boucle6.py
print(list(range(1,5)))

for i in range(1,5):
    print(i)
print("Fin de la boucle")

>>>
[1, 2, 3, 4]
1
2
3
4
Fin de la boucle
```

Table de multiplication

La création d'une table de multiplication paraît plus simple avec une boucle for qu'avec une boucle while :

```
# script Boucle7.py

for compteur in range(1,11):
    print(compteur, '* 9 =', compteur*9)
print("Et voilà !")

>>>
1 * 9 = 9
2 * 9 = 18
3 * 9 = 27
4 * 9 = 36
5 * 9 = 45
6 * 9 = 54
7 * 9 = 63
8 * 9 = 72
9 * 9 = 81
10 * 9 = 90
Et voilà !
```

III) <u>POUR ARRETER UNE BOUCLE : L'INSTRUCTION BREAK</u> :

L'instruction break provoque une sortie immédiate d'une boucle while ou d'une boucle for. Dans l'exemple suivant, l'expression True est toujours ... vraie : on a une boucle sans fin. L'instruction break est donc le seul moyen de sortir de la boucle.

Affichage de l'heure courante

```
# script Boucle8.py

import time     # importation du module time
```

```
while True:
    # strftime() est une fonction du module time
    print('Heure courante ', time.strftime('%H:%M:%S'))
    quitter = input('Voulez-vous quitter le programme (o/n) ? ')
    if quitter == 'o':
        break
print("A bientôt")

>>>
Heure courante  14:25:12
Voulez-vous quitter le programme (o/n) ? n
Heure courante  14:25:20
Voulez-vous quitter le programme (o/n) ? o
A bientôt
```

Astuce
Si vous connaissez le nombre de boucles à effectuer, utiliser une boucle for.
Autrement, utiliser une boucle while (notamment pour faire des boucles sans fin).

Exercice 3 : Écrire un programme qui affiche un rectangle de *n* lignes sur *p* colonnes rempli de caractères * sur le modèle ci-dessous (pour $n = 3$ et $p = 5$).

```
*****
*****
*****
```

Écrire ensuite un programme qui affiche un triangle rectangle isocèle de taille *n* constitué de caractères * sur le modèle ci-dessous (pour $n = 3$).

```
*
**
***
```

IV) APPLICATIONS :

Exercice 4 :
Écrivez un programme qui affiche les 20 premiers termes de la table de multiplication par 7, en signalant au passage (à l'aide d'un astérisque) ceux qui sont des multiples de 3.
Exemple : 7 14 21 * 28 35 42 * 49 ...

Exercice 5 : ***plus difficile…et pour s'entrainer…***
Cryptographie ou l'art de dissimuler de l'information
Méthode inverse
Cette technique très rudimentaire permet d'afficher le message dans l'ordre inverse. Par exemple bonjour devient ruojnob. Pour décoder ce message il suffit à nouveau de l'afficher dans l'ordre inverse. Le codage et le décodage du message obéissent à la même stratégie, on dira au même **algorithme**.
1°) Rédiger à l'aide d'une boucle conditionnelle le code permettant de réaliser le cryptage inverse d'un message. Pour déterminer la longueur d'une chaîne de caractères on utilise la fonction len par exemple :
len("bonjour") renvoie la valeur entière (de type int) égale à 7.
2°) Quelle est la condition d'arrêt de la boucle ?
3°) Évaluer le nombre d'itérations nécessaires pour effectuer ce cryptage.
4°) Tester le codage et le décodage sur l'ordinateur.
5°) À l'aide de ce code comment vérifier si une chaîne de caractères est un palindrome ?
Le palindrome (substantif masculin), aussi appelé palindrome de lettres, est une figure de style désignant un texte ou un mot dont l'ordre des lettres reste le même qu'on le lise de gauche à droite ou de droite à gauche, comme dans la phrase "Ésope reste ici et se repose" ou encore "La mariée ira mal" à un accent près.

Chiffre de César
Chiffrement par décalage avec débordement
En cryptographie le chiffre de César est aussi connu sous le nom de chiffrement par décalage. Cette méthode a été utilisée par Jules César pour ses correspondances secrètes. Le texte crypté s'obtient en remplaçant chaque lettre du message original par une lettre à distance fixe et toujours du même coté dans l'ordre de l'alphabet. Il s'agit d'une

permutation circulaire de l'alphabet. Par exemple un décalage de 2 vers la droite implique que A est remplacé par C, B devient D, C devient E et ainsi de suite jusqu'à Y qui devient A et enfin Z qui devient B.
1°) Tester le code suivant et indiquer l'utilité de la fonction ord. On pourra faire une recherche internet sur la notion de code ASCII standard et étendu.

```
alphabet = "abcdefghijklmnopqrstuvwxyz"
for lettre in alphabet:
        print (ord(lettre))
```

2°) La fonction ord est la primitive inverse de chr. C'est à dire que l'instruction chr(97) renvoie la lettre a.
Rédiger une suite d'instructions permettant d'obtenir une chaîne de caractères contenant toutes les lettres de l'alphabet en majuscule à partir de la fonction chr.
3°) Dans un premier temps nous allons crypter un message en respectant uniquement le principe de chiffrement par décalage. Nous ne nous occuperons pas de savoir si le caractère crypté est une lettre ou non. C'est-à-dire que nous ne tiendrons pas compte de la propriété circulaire du décalage.
Rédiger le code permettant d'obtenir une substitution des caractères du message en clair avec un décalage de trois. *Ce décalage est celui que César a utilisé pour crypter ses messages.*
4°) Tester votre code avec la phrase suivante : *Voyez vous ce que je vois ?*
Résultat :
Yr|h}#yrxv#fh#txh#mh#yrlv#B
5°) Faire un nouveau test avec la phrase : Je soussigné ... atteste avoir réussi la première partie du TP

<u>Algorithmique et programmation</u>
Chiffrement par décalage sans débordement
Le décalage est en fait une permutation circulaire sur les lettres de l'alphabet, le cryptage d'une lettre en clair est forcément une lettre. Pour simplifier nous allons considérer que toutes les lettres du message sont en minuscule et qu'il ne devra pas contenir de caractères accentués, pour cela il pourra être rédigé en anglais ou bien les caractères accentués seront remplacés par les mêmes caractères mais sans accent.
1°) Écrire une suite d'instructions permettant de ne pas afficher la ponctuation contenue dans un texte.
Exemple :
Elle cria: "Je prends, je rachète tout ça. Ce que vous vendez là, c'est mon passé à moi..."
Elle cria Je prends je rachète tout ça Ce que vous vendez là c'est mon passé à moi
2°) Il s'agit maintenant de supprimer les caractères spéciaux. Il est vivement conseillé d'utiliser la **méthode : .replace** que possède un objet de type string.
Elle cria Je prends je rachete tout ca Ce que vous vendez la c'est mon passe a moi
Exemple d'utilisation de la documentation liée à Python

```
>>>
help("".replace)
replace(...)
S.replace(old, new[, count]) -> string
Return a copy of string S with all occurrences of substring
old replaced by new. If the optional argument count is
given, only the first count occurrences are replaced.
```

3°) À l'aide de l'instruction : print (dir("")) trouver et tester une fonction pour convertir tous les caractères d'une chaîne en minuscule.
4°) Sachant que ord("a") = 97 et que ord("A") = 65 proposer une autre méthode pour convertir toutes les lettres de notre message en minuscule. Aucun code n'est demandé mais les plus courageux pourront le faire…
5°) Pour terminer, rédiger une suite d'instructions permettant d'obtenir un chiffrement par décalage avec permutation circulaire et une clé de 20.
<u>Exemple</u> :
ayé ? GG, Xd :) -> usy aa rx
6°) Tester votre code.
Décodage du message crypté
<u>Avec la clé</u>
Connaissant la clé (20), rédiger une suite d'instructions pour décoder ce message.
wy koy hiom xypihm ujjlyhxly u zucly, hiom fujjlyhihm yh fy zucmuhn
<u>Sans la clé : méthode de la force brute</u>
Proposer une solution permettant de décrypter le message précédent si on ne possède pas la clé de codage.